REA(Reverse Engineer Anything) 바이럴 게시물 분석 보고서

Oct 13, 2026 · @chunillkkoon

요약

REA는 실재하는 오픈소스 도구입니다. 그러나 게시물의 "TradingView 프리미엄 지표를 공짜로 쓴다"는 주장은 근거가 없고, REA의 공식 이용 원칙과도 정면으로 충돌합니다. 이 게시물은 실재하는 인기 도구에 유명 브랜드와 금기 위반 암시를 결합한 참여 유도형 콘텐츠로 판단됩니다.

구분	판단
REA의 실재 여부	확인됨. GitHub 저장소 `morluto/rea`와 npm 패키지 `rea-agents` v6.4.0이 스크린샷과 일치합니다.
도구의 성격	새로운 기술이 아니라 Ghidra·Hopper·IDA 같은 기존 분석 도구를 AI 에이전트에 연결해 주는 CLI·MCP 서버입니다.
"프리미엄 지표 공짜 사용"	근거 없음. REA README가 라이선스 우회를 허가되지 않은 용도로 명시합니다.
본문 8단계의 실체	해킹이 아니라, 관찰한 동작을 참고해 비슷한 앱을 새로 만드는 클린룸 재구현입니다.
게시물 성격	참여 유도형 바이럴 콘텐츠입니다.
실무적 가치	개인이 AI로 퀀트 분석 터미널을 만드는 것은 현실적으로 가능합니다. 다만 리버스 엔지니어링은 거의 필요 없습니다.

게시물 원문 번역

분석 대상은 Tauhid IQ 계정이 2026년 10월 초에 올린 게시물입니다(캐프처 시점 기준 14시간 전). 다음은 본문 번역입니다.

> 이게 지금 인터넷에서 가장 **위험한** 것이다.
>
> REA(Reverse Engineer Anything): 이 오픈소스 MCP를 쓰면 AI 에이전트가 어떤 앱이든 바이너리 수준까지 리버스 엔지니어링할 수 있다.
>
> 방금 TradingView를 리버스 엔지니어링했고, 지금 그들의 프리미엄 지표를 **공짜로** 쓰고 있다. 소프트웨어 산업은 끝났다(software is cooked).
>
> REA로 TradingView를 리버스 엔지니어링하는 방법:
>
> 1. TradingView의 공개적으로 관찰 가능한 인터페이스에 REA를 연결하고, 차트 조작부터 지표·알림까지 주요 기능을 전부 정리한다.
> 2. 종목이나 타임프레임을 바꾸고, 지표를 추가하고, 레벨을 그리고, 차트 레이아웃을 바꿀 때 인터페이스가 어떻게 반응하는지 추적한다.
> 3. REA의 MCP 도구로 허가된 소프트웨어 산출물을 조사하고, 증거가 허락하는 범위 안에서 애플리케이션 로직을 추적한다.
> 4. 조사 결과를 Claude Opus 5.5에 넣으면 기술 명세서와 구현 계획으로 정리해 준다.
> 5. 실제 시장 데이터, 캔들스틱, 기술 지표, 드로잉 도구, 저장되는 관심종목을 갖춘 차트 엔진을 독자적으로 다시 만든다.
> 6. 수수료·슬리피지·낙폭(drawdown)을 반영해 과거 데이터로 전략을 검증하는 퀀트 백테스트 엔진을 붙인다.
> 7. 모든 구성요소를 테스트하고, 문서와 관찰 결과에 비춰 동작을 비교하고, 실패를 고치며 시스템이 실제로 돌아갈 때까지 반복한다.
> 8. 결국 소스코드를 직접 소유하고 계속 확장할 수 있는, TradingView 스타일의 퀀트 리서치 터미널을 처음부터 직접 만들게 된다.
>
> 더 이상은 말하지 않겠다. 업데이트를 보려면 Tauhid IQ를 꼭 팔로우하세요.

첨부 이미지는 세 부분으로 구성됩니다.

이미지	내용
GitHub 저장소 화면	브랜치 66개, 태그 35개, 커밋 2,349개. 폴더는 `.agents/skills`, `bridge`, `docs` 등. 토픽은 ghidra, hopper, decompiler, mcp, ctf 등.
README 배너	"바이너리, 애플리케이션, 런타임 동작 전반을 리버스 엔지니어링하는 하나의 MCP." 배지는 npm v6.4.0, CI failing, MIT, Discord 1.5k online, "GitHub Trending #1".
차트 스크린샷	TradingView 로고와 BTCUSDT 5분봉. LuxAlgo의 "Ultimate RSI"와 "Institutional Momentum Zones" 지표, 암호화폐·외환·지수 관심종목 목록.

게시물 반응은 공감 132, 댓글 14, 공유 74, 저장 296입니다.

REA 실체 검증

REA는 2026년 7월 12일 처음 배포된 MIT 라이선스 오픈소스입니다. 스크린샷의 저장소·배지 정보가 실제 데이터와 일치합니다.

항목	확인 결과	출처
저장소	`morluto/rea`. 소개 문구가 스크린샷과 같습니다.	GitHub
개발자	`morluto` (npm 관리자도 동일)	GitHub, npm
라이선스	MIT	GitHub
GitHub 지표	스타 약 17.7만, 포크 약 4.25만 (조회 시점 기준)	GitHub
npm 패키지	`rea-agents` 최신 버전 6.4.0. 스크린샷 배지와 일치합니다.	npm 레지스트리
최초 배포	2026-07-12. 구 패키지 `@morluto/rea` 0.2.1도 같은 날 배포 후 중단됐습니다.	npm 레지스트리
최근 배포	2026-10-11. 약 3개월 동안 31개 버전을 냈습니다.	npm 레지스트리
설치 스크립트	postinstall·install 스크립트 없음. 설치만으로 코드가 자동 실행되지 않습니다.	npm 레지스트리
실행 환경	Node.js 22.19 이상. 분석 대상에 따라 Ghidra, Hopper, IDA, adb, tshark 등이 따로 필요합니다.	GitHub README

실제 기능 범위

REA는 AI 코딩 에이전트(Claude Code, Codex 등)와 기존 리버스 엔지니어링 도구를 잇는 연결 계층입니다.

- **분석 대상**: 네이티브 바이너리, JavaScript/Electron 앱, .NET 어셈블리, 웹사이트, Go 바이너리, WASM, EVM 바이트코드, SQLite, 안드로이드 APK, Flutter 앱, 펌웨어, 패킷 캡처
- **분석 엔진**: Ghidra, Hopper, IDA. JavaScript와 .NET 정적 분석은 별도 엔진이 필요 없습니다.
- **결과물**: 의사코드, 어셈블리, 문자열, 심볼, 호출·참조 관계, 모듈, IPC 관계. 결론마다 근거와 한계를 함께 붙입니다.

MindStudio 리뷰는 REA를 "새로운 기술이 아닌 연결 도구"로 평가했습니다. 일반 에이전트로도 비슷한 작업이 가능하고, 과열의 상당 부분은 GitHub 스타 증가가 이끈다고 봤습니다. 이 리뷰는 네트워크 제한으로 직접 열지 못하고 검색 요약으로 확인했습니다.

게시물 주장 분석

게시물의 헤드라인은 "TradingView를 뚫었다"고 말하지만, 본문 단계는 "관찰해서 새로 만들었다"는 내용입니다. 두 주장은 서로 맞지 않습니다.

제목과 본문의 불일치

본문에는 "공개적으로 관찰 가능한 인터페이스", "허가된 소프트웨어 산출물", "증거가 허락하는 범위", "독자적으로 다시 만든다" 같은 법적으로 조심스러운 표현이 나옵니다. 이 문구들은 REA README의 허용 범위 문구와 맞아떨어집니다. README 표현을 가져와 놓고 헤드라인만 자극적으로 바꾼 것으로 보입니다.

게시물 주장과 REA 공식 입장 대조

게시물 주장	REA README의 공식 입장
"프리미엄 지표를 공짜로 쓴다"	라이선스 우회는 허가된 용도가 아니라고 명시합니다.
"어떤 앱이든 리버스 엔지니어링"	본인 소유이거나 분석 권한을 받은 소프트웨어만 대상으로 하라고 합니다.
"가장 위험한 것"	합법적 연구, 교육, 상호운용성, 소프트웨어 보존 목적이라고 밝힙니다.
—	상용 분석 도구도 합법적으로 구한 것만 쓰고, 크랙 버전과 도난 키는 쓰지 말라고 합니다.

REA README 어디에도 TradingView는 언급되지 않습니다.

"프리미엄 지표 공짜 사용" 주장의 기술적 검토

1. **스크린샷의 지표는 TradingView 것이 아닙니다.** "Ultimate RSI"와 "Institutional Momentum Zones"는 제3자 판매사 LuxAlgo의 지표입니다. LuxAlgo는 무료 공개 지표도 많이 올려 둡니다. 이 화면만으로는 유료 기능 우회의 증거가 되지 않습니다.
2. **스크린샷은 진짜 TradingView 화면일 가능성이 큽니다.** TradingView 워터마크, "Publish" 버튼, "Unnamed" 레이아웃 표시는 실제 웹 앱의 UI 요소입니다. 그렇다면 "내가 만든 터미널"의 증거로도 쓸 수 없습니다.
3. **비공개 지표 로직은 클라이언트에 없을 가능성이 큽니다.** 일반적으로 알려진 바로는, 보호(protected)·초대 전용(invite-only) Pine Script는 원본 소스가 브라우저로 전달되지 않습니다. 앱을 분석해도 출력값을 보고 비슷하게 근사하는 정도가 한계입니다.
4. **표준 지표는 원래 공개된 수학입니다.** RSI, EMA, MACD, 볼린저 밴드는 공식이 공개돼 있어 리버스 엔지니어링이 필요 없습니다.

참고로 폐쇄형 TradingView 지표를 AI로 복제하는 시도는 REA 이전부터 있었습니다. 차트 데이터(OHLC)와 지표 출력값을 뽑아 AI에게 공식을 추정하게 하는 영상 튜토리얼이 대표적입니다. 이 방식도 원본 코드를 얻는 것이 아니라 출력을 보고 비슷한 공식을 맞추는 것입니다.

8단계 방법론의 현실성

과장을 걷어내면 8단계는 현실적인 개발 계획입니다. 다만 핵심인 3단계(바이너리 분석)는 웹 앱을 복제하는 데 거의 필요하지 않습니다.

단계	현실성	비고
1~2. UI 기능 정리, 동작 관찰	높음	사실상 제품 분석·기획 작업입니다. 브라우저 자동화면 충분합니다.
3. 바이너리 분석	필요성 낮음	웹 앱 클론에 디컴파일러는 거의 필요 없습니다. 홍보를 위한 연결로 보입니다.
4. LLM으로 명세·계획 작성	높음	현재 AI 코딩 에이전트가 잘하는 영역입니다.
5. 차트 엔진	중간~높음	TradingView가 직접 공개한 오픈소스 Lightweight Charts로 시작할 수 있습니다.
5. 시장 데이터	주의 필요	암호화폐는 거래소 공개 API로 무료지만, 주식·지수 실시간 데이터는 라이선스 비용이 큽니다.
6. 백테스트 엔진	중간	backtrader, vectorbt 등 기존 도구가 있습니다. 미래 데이터 참조(look-ahead bias)와 과최적화 방지가 어렵습니다.
7~8. 테스트와 반복 개선	높음	일반적인 개발 과정입니다.

TradingView의 경쟁력은 코드가 아니라 데이터 라이선스, 커뮤니티 스크립트 생태계, 서비스 안정성에 있습니다. 그래서 "software is cooked"는 과장입니다.

바이럴 구조와 수치 해석

이 게시물은 참여 유도형 콘텐츠의 전형적인 장치를 대부분 갖춰습니다.

장치	실제 문구	목적
공포와 충격	"most DANGEROUS thing on internet"	스크롤 멈추게 하기
금기 위반 암시	"premium indicators for FREE"	호기심과 이득 욕구 자극
거대 담론	"software is cooked"	공유와 논쟁 유도

장치	실제 문구	목적
단계별 리스트	1~8단계	"저장" 유도
신비주의 마무리	"i won't say anything more"	궁금증 남기기
팔로우 요구	"Must Follow"	계정 성장
유명 브랜드 결합	TradingView 로고, Claude 모델명	신뢰도 차용

수치 해석

- **저장 296건이 공감 132건의 2배가 넘습니다.** "나중에 따라 해 볼 정보"로 소비되고 있다는 뜻이고, 그만큼 과장이 사실로 받아들여질 위험이 큽니다.
- **3개월 만에 GitHub 스타 약 17.7만 개**는 극히 이례적인 속도입니다. 스타는 조작이 가능한 지표라 품질의 근거로 쓰면 안 됩니다.
- **3개월 동안 31개 버전, 메이저 버전 6**은 개발이 활발하다는 뜻이면서 아직 불안정하다는 뜻입니다. 스크린샷의 "CI failing" 배지와도 맞습니다.
- **"GitHub Trending #1" 배지**는 누구나 이미지로 만들어 붙일 수 있어 검증된 지표가 아닙니다.
- **MCP 도구 수**는 소스마다 42·50·70개로 다릅니다. 버전마다 도구가 늘어났기 때문으로 보입니다.

법적·윤리적·보안 리스크

게시물처럼 상용 서비스의 유료 기능을 우회하면 약관 위반과 저작권 침해 소지가 큽니다. 반면 공개 공식과 오픈소스로 자기 도구를 만드는 것은 대체로 안전합니다. 아래는 일반 정보이며 법률 자문이 아닙니다.

법적·윤리적 리스크

- **이용약관 위반**: TradingView를 포함한 대부분의 SaaS 약관은 리버스 엔지니어링, 무단 스크래핑, 유료 기능 우회를 금지합니다. 계정 정지나 민사 책임으로 이어질 수 있습니다.
- **한국 저작권법**: 프로그램코드역분석 조항(제101조의4)은 호환에 필요한 정보를 얻는 목적만 허용합니다. 그 정보로 유사한 프로그램을 만들어 경쟁하는 데 쓰는 것은 허용되지 않습니다.
- **미국 DMCA**: 기술적 보호조치 우회 금지 조항이 문제될 수 있습니다.
- **제3자 지표 판매사의 권리**: LuxAlgo 같은 판매사의 유료 지표 로직을 빼내 재현하면 저작권이나 영업비밀 침해가 문제될 수 있습니다.
- **허용되는 영역**: 공개된 수학 공식으로 지표를 구현하는 것, 공개 API와 오픈소스로 차트 앱을 만드는 것, UI 아이디어를 참고하는 것은 대체로 안전합니다.

보안 리스크

- **넓은 권한**: REA는 런타임 캡처 시 분석 대상 프로그램을 사용자 권한으로 실행합니다. 의존성에도 터미널 제어(node-pty)와 브라우저 자동화(playwright-core)가 들어 있습니다.

- **프롬프트 인젝션**: 분석 대상 바이너리나 웹페이지에 AI를 노린 문자열이 있으면 에이전트가 그 지시를 따를 수 있습니다.
- **공급망**: 설치 시 자동 실행 스크립트가 없어 위험이 일부 완화됩니다. 다만 배포 주기가 빨라 버전 고정이 필요합니다.
- **사칭 패키지와 크랙 유포**: 유명 도구를 흉내 낸 악성 패키지가 자주 등장합니다. "TradingView 프리미엄 크랙"을 배포한다는 게시물과 스팸 댓글도 실제로 돌고 있습니다. 이런 파일은 악성코드 유포 통로로 자주 쓰입니다.

결론 및 권고

REA는 허가된 범위에서 쓸모 있는 도구이고, 게시물의 "공짜 프리미엄" 주장은 따라 해서는 안 되는 과장입니다.

1. **도구 평가**: REA는 CTF, 보안 연구, 본인 소프트웨어 분석, 상호운용성 연구에 실제로 쓸모가 있어 보입니다.
2. **게시물 평가**: 실제로 설명된 내용은 공개된 동작을 관찰해 비슷한 앱을 새로 만드는 것입니다. "TradingView를 뚫었다"는 주장은 근거가 없습니다.
3. **비슷한 도구를 만들고 싶다면**: 오픈소스 차트(Lightweight Charts), 거래소 공개 API, 공개 지표 공식, 오픈소스 백테스트 라이브러리를 조합하세요. 합법적이고 더 빠릅니다.
4. **REA를 써 본다면**: 공식 패키지 `rea-agents`만 쓰고, 버전을 고정하고, VM이나 컨테이너에서 실행하고, 본인 소유이거나 허가받은 대상에만 쓰세요.
5. **유료 지표가 필요하다면**: 작성자의 무료·오픈소스 버전을 쓰거나, 유료 구독을 하거나, 작성자에게 접근 권한을 요청하세요. "크랙" 파일은 악성코드 위험이 큽니다.

검증 한계

- GitHub 수치는 조회 시점의 웹페이지 기준입니다.
- MindStudio 리뷰, EveryDev, Socket, 텔레그램 게시물은 네트워크 제한으로 직접 열지 못하고 검색 요약으로만 확인했습니다.
- 게시자(Tauhid IQ)가 실제로 TradingView를 분석했는지는 확인할 방법이 없습니다.
- 법적 내용은 일반 정보이며 법률 자문이 아닙니다.

출처

직접 연 페이지:

- morluto/rea (GitHub)
- npm 레지스트리: rea-agents
- npm 레지스트리: @morluto/rea

검색 요약으로만 확인한 페이지:

- MindStudio: Reverse Engineer Anything MCP 리뷰
- EveryDev: REA 소개
- mdskills: REA 스킬
- Socket: @morluto/rea 패키지
- Telegram SkipCI: Reverse engineer any app…
- 폐쇄형 TradingView 지표 AI 복제 영상
- r/BestTrades 크랙 홍보 게시물 미러
- TradingView 내 무료 지표 스팸 사례
- Ehlers Reverse EMA (오픈소스 지표 예시)